K.R. Mangalam University

School of Engineering and Technology

SMART CITY TRAFFIC MANAGEMENT AND ROUTE OPTIMIZATION SYSTEM

Design and Analysis of Algorithms (DAA) Lab 5, Capstone Assignment (Option A)

Program: BCA (AI & DS)

Course Code - ENCA301 (2026-27)

GitHub: akash77007/Smart-City-Traffic-Optimization

Submitted by:

Name: Akash Sharma

Roll Number: 2401201108

Course: BCA (AI & DS) - Section B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor, SOET

1. Introduction

Cities lose time, fuel and money to congested roads, while static traffic plans cannot react to rush-hour surges or sudden closures. A city road network is naturally a weighted graph: intersections are vertices, roads are edges and the weight is the travel time. Smart city systems use graph algorithms to run transportation: shortest-path algorithms route commuters, traversal maps reachable zones, and spanning trees plan the cheapest set of roads that keeps every intersection connected.

Traffic optimization matters because congestion wastes time and fuel, delays emergency response, and underuses existing roads. A system that recomputes routes when roads jam, audits emergency coverage, and budgets maintenance for minimum cost turns the road network from a static map into a managed resource.

This project builds such a system: a 10-intersection test city with 19 roads plus four modules (route planning with Dijkstra and Bellman-Ford, BFS/DFS connectivity, Prim/Kruskal minimum spanning trees), benchmarks on random graphs of 10 to 150 intersections, automated tests, and a Streamlit dashboard with a command-line menu.

Objectives.

- **Graph modelling (Phase 1):** represent the road network with an adjacency list and an adjacency matrix and compare their memory use.
- **Route planning and congestion (Phase 2):** fastest routes with Dijkstra, toll credits and cycle detection with Bellman-Ford, and incident slowdowns with rerouting.
- **Connectivity analysis (Phase 3):** BFS for minimum-hop reach and DFS for components and deep exploration, to audit emergency dispatch coverage.
- **Infrastructure budgeting (Phase 4):** the minimum spanning tree with Prim and Kruskal, and the maintenance saving it gives.
- **Benchmarking:** execution time and peak memory for several graph sizes, and a theoretical vs observed complexity table.
- **Visualization and interfaces (Phase 5):** network, route, tree and comparison charts, a Streamlit dashboard and a command-line menu.

Problem. The city has fixed intersections and roads, but no single tool that finds the fastest route, shows how traffic reroutes when a road jams, audits which intersections are close to every other, and plans the cheapest set of roads to maintain. Planners need those answers on one shared road model, with the cost of each decision visible.

Design overview. All modules share the city data and graph builders from `graph_model.py`. `shortest_path.py` runs Dijkstra, Bellman-Ford and the congestion simulator on that graph, `traversal.py` traverses its adjacency list, `mst.py` consumes its roads, `analysis/benchmark.py` drives every algorithm on seeded random graphs, and `app.py` and `cli.py` expose the same functions to the user.

Environment and tools.

Item	Details
Language	Python 3 (CPython)
Algorithms	algorithms/ package: graph_model.py, shortest_path.py, traversal.py, mst.py

Item	Details
Timing	time.perf_counter() around each call; best of 15 repeated runs per (algorithm, V), best of 5 for Bellman-Ford
Memory	tracemalloc peak traced allocation (KB) in a separate single run per (algorithm, V); the input graph is built before tracing starts
Charts	matplotlib and NetworkX for maps and plots; tables from the exported CSV
Interfaces	Streamlit web dashboard (app.py), menu-driven CLI (cli.py), automated tests (tests/test_algorithms.py)
Reproducibility	random.seed(42); results exported to data_csv/benchmark_results.csv; figures to graphes/
Source repository	github.com/akash77007/Smart-City-Traffic-Optimization

Setup: Python 3, then `pip install streamlit matplotlib networkx` (plus `pip install pandas numpy` if they are not already installed). Work is delivered against the five capstone phases:

Phase	Deliverable	Where
1	Project setup and road network modeling	2.1, repository layout
2	Route planning and traffic analysis	2.2, 2.3, 4.2
3	Network connectivity analysis	2.4, 4.1
4	Infrastructure planning and comparison	2.5, 2.6, 4.3
5	Interface, visualization, testing, report	3, 4, 5, 6

2. Modules

2.1 Road network modeling

Intersections are vertices, roads are weighted undirected edges (travel time in minutes). The test city has 10 intersections and 19 two-way roads of total weight 78, with each intersection tagged by a functional zone and a map coordinate. The model exposes both representations:

graph_model.py (city data and graph builders):

```
NODES = {  # id: (name, zone, (x, y))
    "A": ("Downtown Central", "Commercial", (20, 50)),
    "B": ("North Station", "Transport Hub", (40, 80)),
    "C": ("Tech Park", "Industrial", (40, 20)),
    "D": ("University Campus", "Educational", (70, 80)),
    "E": ("Civic Center", "Government", (65, 50)),
    "F": ("South Market", "Commercial", (70, 20)),
    "G": ("Airport Terminal", "Transport Hub", (95, 70)),
    "H": ("Green Valley", "Residential", (95, 30)),
    "I": ("Harbor Point", "Industrial", (15, 20)),
    "J": ("East Suburbs", "Residential", (90, 50)),
}
EDGES = [  # (u, v, travel time in minutes)
    ("A", "B", 4), ("A", "C", 2), ("A", "I", 3), ("B", "C", 1),
    ("B", "D", 5), ("B", "E", 6), ("C", "D", 8), ("C", "F", 3),
    ("C", "I", 4), ("D", "E", 2), ("D", "G", 6), ("E", "F", 3),
    ("E", "G", 7), ("E", "J", 4), ("F", "H", 6), ("F", "J", 5),
    ("G", "H", 4), ("G", "J", 3), ("H", "J", 2),
]
def build_adjacency_list(edges):
    """O(V + E) memory; undirected, so each road is stored twice."""
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
        graph.setdefault(v, []).append((u, w))
    return graph
def build_adjacency_matrix(nodes, edges):
    """O(V^2) memory; O(1) road lookup."""
    idx = {n: i for i, n in enumerate(nodes)}
    size = len(nodes)
    matrix = [[math.inf] * size for _ in range(size)]
    for i in range(size):
        matrix[i][i] = 0
    for u, v, w in edges:
        matrix[idx[u]][idx[v]] = w
        matrix[idx[v]][idx[u]] = w
    return matrix
```

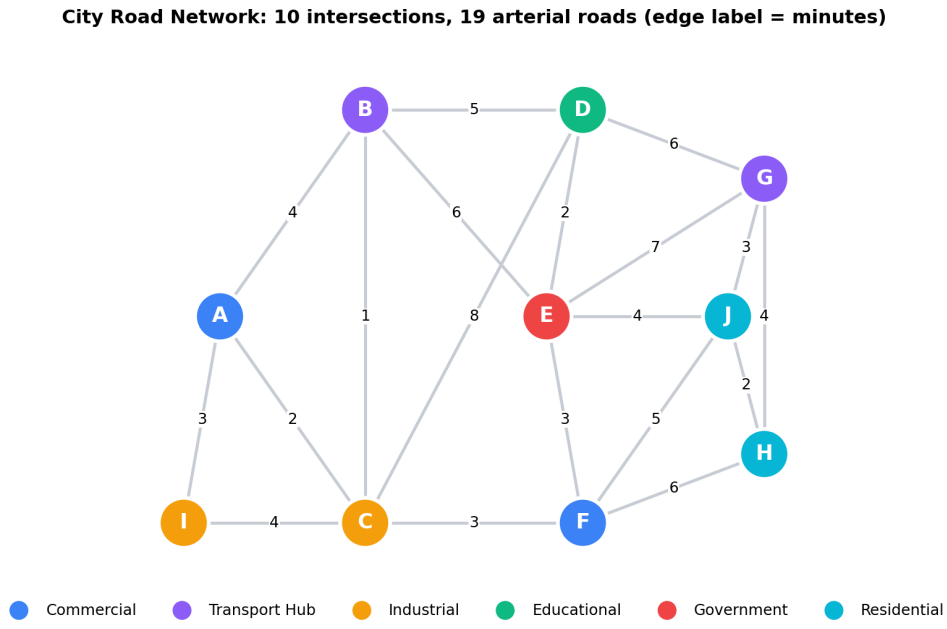
Running the builders on the city produces the node list, the road count and the adjacency list:

```
$ python demo.py
NODES: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
ROADS: 19   TOTAL WEIGHT: 78
ADJACENCY LIST:
  A: [('B', 4), ('C', 2), ('I', 3)]
  B: [('A', 4), ('C', 1), ('D', 5), ('E', 6)]
  C: [('A', 2), ('B', 1), ('D', 8), ('F', 3), ('I', 4)]
  D: [('B', 5), ('C', 8), ('E', 2), ('G', 6)]
  E: [('B', 6), ('D', 2), ('F', 3), ('G', 7), ('J', 4)]
  F: [('C', 3), ('E', 3), ('H', 6), ('J', 5)]
  G: [('D', 6), ('E', 7), ('H', 4), ('J', 3)]
  H: [('F', 6), ('G', 4), ('J', 2)]
  I: [('A', 3), ('C', 4)]
  J: [('E', 4), ('F', 5), ('G', 3), ('H', 2)]
```

```
$ python demo.py   # adjacency matrix (0 = diagonal, . = no road)
       A   B   C   D   E   F   G   H   I   J
  A    0   4   2   .   .   .   .   .   3   .
  B    4   0   1   5   6   .   .   .   .   .
```

```
    C     2   1   0   8   .   3   .   .   4   .
    D     .   5   8   0   2   .   6   .   .   .
    E     .   6   .   2   0   3   7   .   .   4
    F     .   .   3   .   3   0   .   6   .   5
    G     .   .   .   6   7   .   0   4   .   3
    H     .   .   .   .   .   6   4   0   .   2
    I     3   .   4   .   .   .   .   .   0   .
    J     .   .   .   .   4   5   3   2   .   0
MATRIX: 10 x 10 = 100 cells; list holds 38 directed entries
```

The matrix stores 100 cells whatever the number of roads, while the adjacency list holds 2E = 38 directed entries for 19 undirected roads. Only 19 of the 45 possible roads exist, so the network is sparse and the list is the natural default; the matrix is $O(V^2)$ and only wins for dense graphs or constant-time road tests.



City road network: node colour is the functional zone and edge labels are travel times in minutes.

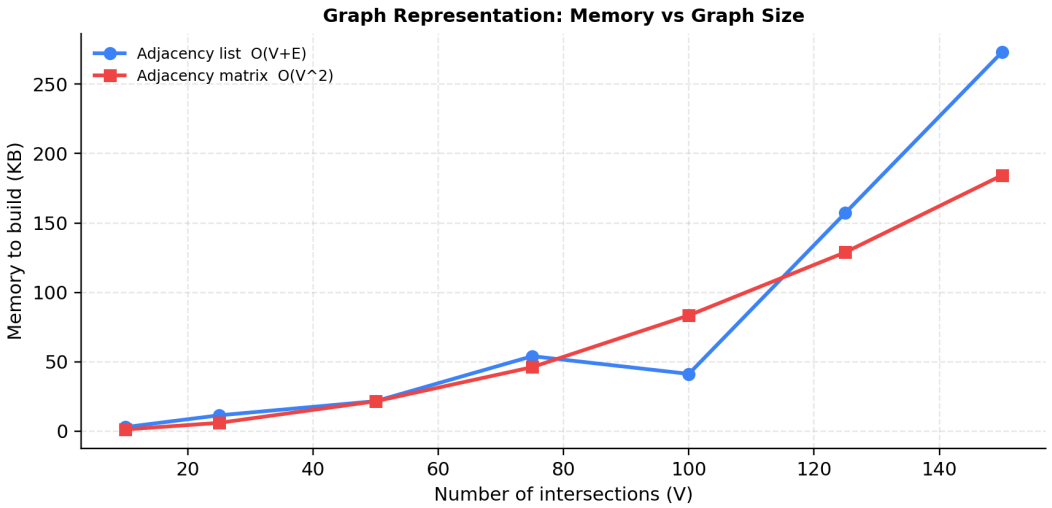
ID	Intersection	Zone	(X, Y)
A	Downtown Central	Commercial	(20, 50)
B	North Station	Transport Hub	(40, 80)
C	Tech Park	Industrial	(40, 20)
D	University Campus	Educational	(70, 80)
E	Civic Center	Government	(65, 50)
F	South Market	Commercial	(70, 20)
G	Airport Terminal	Transport Hub	(95, 70)
H	Green Valley	Residential	(95, 30)
I	Harbor Point	Industrial	(15, 20)
J	East Suburbs	Residential	(90, 50)

Road	Min	Road	Min	Road	Min
A - B	4.0	A - C	2.0	A - I	3.0

Road	Min	Road	Min	Road	Min
B - C	1.0	B - D	5.0	B - E	6.0
C - D	8.0	C - F	3.0	C - I	4.0
D - E	2.0	D - G	6.0	E - F	3.0
E - G	7.0	E - J	4.0	F - H	6.0
F - J	5.0	G - H	4.0	G - J	3.0
H - J	2.0				

The 19 roads above add up to 78 units. The graph splits into two halves: A, B, C and I form the western corridor and D, E, G, H, J the eastern and airport side, linked by B-D, B-E, C-D, C-F and E-F.

Adjacency list versus adjacency matrix (memory)



Memory needed to build the adjacency list and the adjacency matrix.

- At V = 150 the list took 273 KB and the matrix 184 KB. The matrix grows as V^2 whatever the number of roads and the list as V + E, but each Python list entry is a tuple object, so the list only wins when roads are few. On the benchmark graphs E reaches about 26% of the maximum, so the two are close.
- The real city graph is sparse (19 of 45 roads), where the list is the better choice; the matrix remains useful for O(1) road lookups in dense downtown cores.

2.2 Route planning (Dijkstra)

A commuter enters a source and a destination; the system settles intersections in order of current-best distance and returns the shortest route and its cost. Stale heap entries are skipped, and the search stops as soon as the target is settled.

```
$ python demo.py
DIJKSTRA A -> D: A -> C -> B -> D  cost=8
DISTANCES from A: A=0  C=2  B=3  I=3  F=5  D=8  E=8  J=10  H=11  G=13
```

shortest_path.py (Dijkstra):

```
def dijkstra(graph, source, target=None):
    """Min-heap Dijkstra. Time O((V + E) log V), space O(V + E)."""
    dist = {v: math.inf for v in graph}
    prev = {v: None for v in graph}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue  # stale heap entry
        if u == target:
            break
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                prev[v] = u
                heapq.heappush(heap, (dist[v], v))
    return dist, prev
def reconstruct_path(prev, target):
    path = []
    while target is not None:
        path.append(target)
        target = prev[target]
```

From Downtown Central (A) to University Campus (D) the system returns A -> C -> B -> D with cost 2 + 1 + 5 = 8 minutes. Dijkstra is the fastest exact router for positive travel times, but its greedy choice breaks with negative weights.

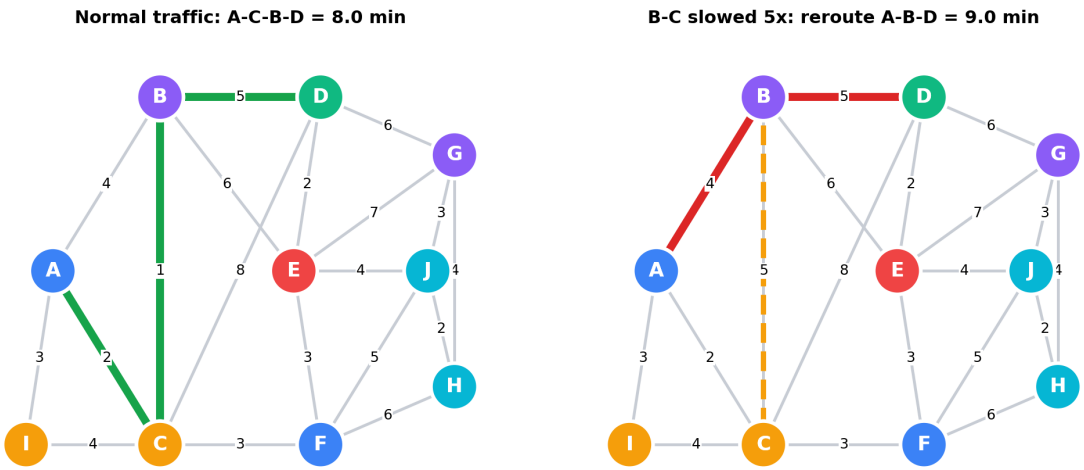
2.3 Traffic analysis (Bellman-Ford plus congestion)

Administrators scale selected road weights to simulate an incident, then Dijkstra is re-run to see whether the best route changes. Bellman-Ford runs on directed arcs, so it also supports one-way toll credits (negative weights) and detects a negative cycle with one extra pass.

shortest_path.py (Bellman-Ford and congestion simulator):

```
def bellman_ford(nodes, edges, source):
    """Directed arcs (u, v, w); negative weights allowed. Time O(V * E).
    Returns (dist, has_negative_cycle). Stops early when a pass changes
    nothing; one extra pass detects a negative cycle.
    """
    dist = {v: math.inf for v in nodes}
    dist[source] = 0
    for _ in range(len(nodes) - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    negative_cycle = any(dist[u] + w < dist[v] for u, v, w in edges)
    return dist, negative_cycle
def simulate_congestion(edges, congested, factor):
    """Multiply the weight of the congested roads by `factor`."""
    hit = {frozenset(pair) for pair in congested}
    return [(u, v, w * factor if frozenset((u, v)) in hit else w)
            for u, v, w in edges]
```

Slowing B-C five times moves A to D from A -> C -> B -> D to A -> B -> D, cost 8 to 9. The table runs all four incident scenarios. Two reroute and two leave the route unchanged:



Route from A to D before and after road B-C is slowed five times (dashed orange).

Trip	Incident	Normal route	Normal (min)	Route kept (min)	New route	New (min)	Rerouted?
A to D	Rush hour: B-C x5	A - C - B - D	8.0	12.0	A - B - D	9.0	Yes
A to G	Freeway repairs: D-E x4	A - C - F - J - G	13.0	13.0	A - C - F - J - G	13.0	No
C to G	Expressway works: D-G x3	C - F - J - G	11.0	11.0	C - F - J - G	11.0	No
F to H	Valley flood: F-H x3	F - H	6.0	18.0	F - J - H	7.0	Yes

- **Rush hour, A to D.** Keeping A-C-B-D would take 2 + 5 + 5 = 12 min; the simulator moves traffic to A-B-D at 9 min, saving 3 min per vehicle.
- **Valley flood, F to H.** The flooded road would cost 18 min; the detour F-J-H takes 7 min, saving 11 min.
- **Unaffected trips.** A to G (A-C-F-J-G, 13 min) and C to G (C-F-J-G, 11 min) never use D-E or D-G, so those incidents change nothing; rerouting should only trigger when the active route is hit.
- **Toll credits.** A one-way credit of -2 on A to I (return direction 3) gives Bellman-Ford a valid distance of -2 to I and no cycle. Making the same road two-way at -2 creates a cycle of weight -4, which the extra pass detects. Credits are therefore only safe on one-way roads.

```
$ python demo.py   # Bellman-Ford toll-credit check
ONE-WAY CREDIT  A->I = -2, I->A = 3 : dist(I) = -2, negative cycle = False
TWO-WAY CREDIT  A-I = -2 both ways  : negative cycle = True
```

2.4 Connectivity (BFS plus DFS)

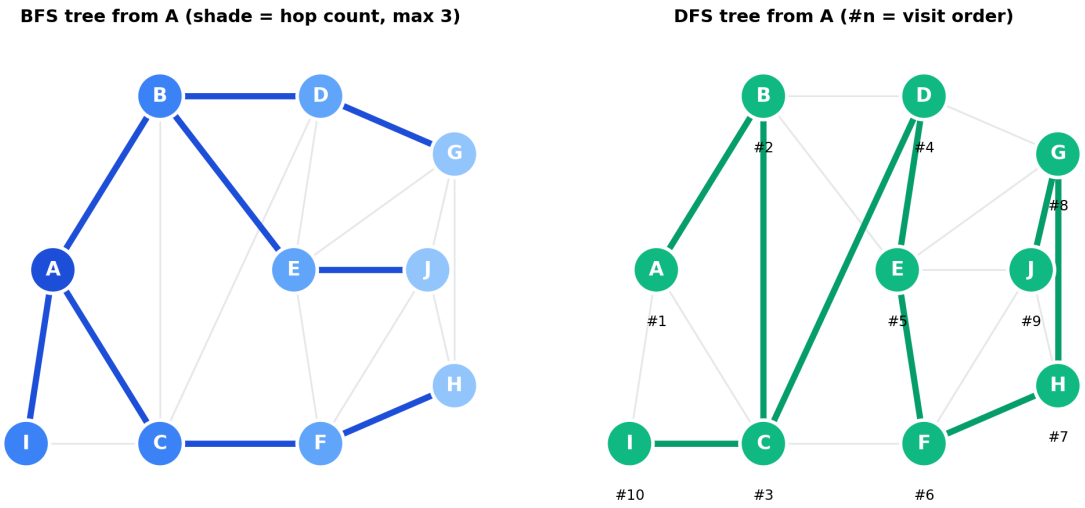
From any start intersection the system lists the traversal order, the hop level of every node and the tree edges that first discover each node. BFS uses a queue and gives true minimum-hop paths; DFS follows one branch as deep as possible.

```
$ python demo.py   # traversal from A
```

```
BFS order: A -> B -> C -> I -> D -> E -> F -> G -> J -> H
BFS tree : A-B, A-C, A-I, B-D, B-E, C-F, D-G, E-J, F-H
BFS hops : A=0  B=1 C=1 I=1  D=2 E=2 F=2  G=3 J=3 H=3
DFS order: A -> B -> C -> D -> E -> F -> H -> G -> J -> I
DFS tree : A-B, B-C, C-D, D-E, E-F, F-H, H-G, G-J, C-I
COMPONENTS: 1 (10 of 10 intersections reachable)
```

traversal.py (BFS, DFS and components):

```
def bfs(graph, source):
    """Returns (visit order, hop level of each node, tree edges)."""
    order, levels, tree = [source], {source: 0}, []
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, _ in graph[u]:
            if v not in levels:
                levels[v] = levels[u] + 1
                tree.append((u, v))
                order.append(v)
                queue.append(v)
    return order, levels, tree
def dfs(graph, source):
    """Recursive DFS. Returns (visit order, tree edges)."""
    order, tree, seen = [], [], set()
    def visit(u):
        seen.add(u)
        order.append(u)
        for v, _ in graph[u]:
            if v not in seen:
                tree.append((u, v))
                visit(v)
    visit(source)
    return order, tree
def connected_components(graph):
    seen, components = set(), []
    for start in graph:
        if start not in seen:
            order, _ = dfs(graph, start)
            seen.update(order)
            components.append(order)
    return components
```



BFS and DFS trees from Downtown Central (A); BFS shading shows hop count, DFS labels show visit order.

Hop	Intersections reached from A
0	A (Downtown Central)
1	B (North Station), C (Tech Park), I (Harbor Point)

Hop	Intersections reached from A
2	D (University Campus), E (Civic Center), F (South Market)
3	G (Airport Terminal), J (East Suburbs), H (Green Valley)

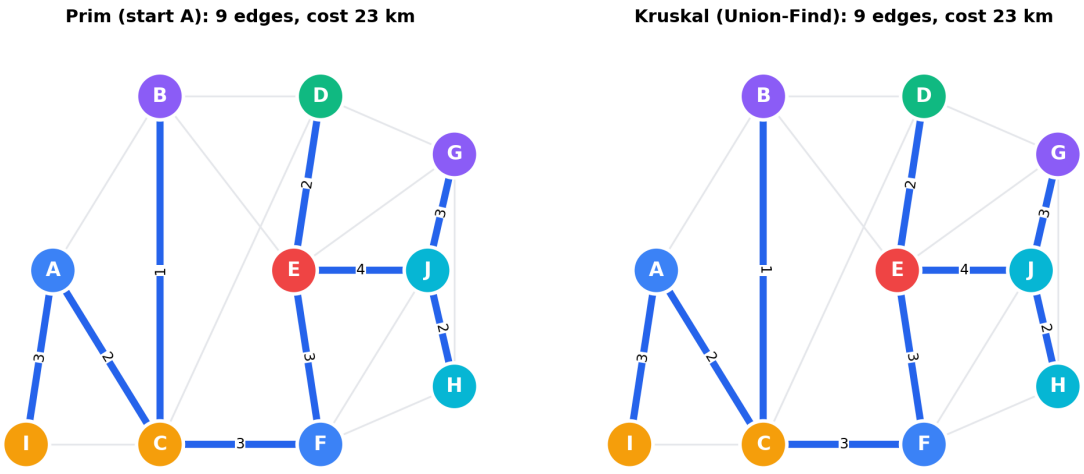
Property	BFS	DFS
Frontier	queue (FIFO)	recursion (call stack)
Visit pattern	level by level	deep branch first
Tree shape	short and wide	long and thin
Furthest from A	3 hops (G, J, H)	visits I last (10th)
Tree edges	9	9

- **Fully connected.** All 10 of 10 intersections are reachable from A; the farthest (G, J and H) are 3 hops away.
- **Tree shape.** DFS visits A - B - C - D - E - F - H - G - J - I; it runs deep first, so its tree is long and thin, while the BFS tree is short and wide and gives the true shortest-hop paths. Both trees have V - 1 = 9 edges because the city is connected.
- **Dispatch sites.** BFS from every intersection shows that C, D and F reach all others within 2 hops; the other seven need up to 3. Those three are the best places for an ambulance or fire station.

2.5 Infrastructure planning (Prim plus Kruskal)

Both MST builders run on the same network and must agree; both select 9 roads totalling 23 minutes. The tree is a maintenance backbone: only these roads need to be kept in top condition to leave every intersection connected.

```
$ python demo.py   # minimum spanning tree
PRIM (start A):   A-C 2, C-B 1, A-I 3, C-F 3, F-E 3, E-D 2, E-J 4, J-H 2, J-G 3
PRIM TOTAL: 23
KRUSKAL:          B-C 1, A-C 2, D-E 2, H-J 2, A-I 3, C-F 3, E-F 3, G-J 3, E-J 4
KRUSKAL TOTAL: 23   SAME TOTAL: True
```



MST found by Prim (left) and Kruskal (right); selected roads are blue.

#	Road	Weight
1	B - C	1
2	A - C	2
3	D - E	2
4	H - J	2
5	A - I	3
6	C - F	3
7	E - F	3
8	G - J	3
9	E - J	4

mst.py (UnionFind plus builders):

```
class UnionFind:
    """Path compression + union by rank."""
    def __init__(self, items):
        self.parent = {x: x for x in items}
        self.rank = {x: 0 for x in items}
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
def kruskal(nodes, edges):
    """Sort edges, add each edge that joins two components. O(E log E)."""
    uf, tree, cost = UnionFind(nodes), [], 0
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        if uf.union(u, v):
            tree.append((u, v, w))
            cost += w
    return tree, cost
def prim(graph, start):
    """Grow one tree with a min-heap of frontier edges. O(E log V)."""
    visited, tree, cost = {start}, [], 0
    heap = [(w, start, v) for v, w in graph[start]]
    heapq.heapify(heap)
    while heap and len(visited) < len(graph):
        w, u, v = heapq.heappop(heap)
        if v in visited:
            continue
        visited.add(v)
        tree.append((u, v, w))
        cost += w
        for nxt, ww in graph[v]:
            if nxt not in visited:
                heapq.heappush(heap, (ww, v, nxt))
    return tree, cost
```

- **Same cost.** Both algorithms return 9 roads costing 23, and the same set of roads.
- **Budget saving.** The full network weighs 78, so maintaining only the MST covers 23: a reduction of (78 - 23) / 78 = 70.5%. The other 10 roads form the secondary network.

- **An MST is not a shortest-path tree.** Inside the MST the only route from A to D is A-C-F-E-D with weight 10, while the full network offers 8 (A-C-B-D). The MST is a maintenance backbone, not a routing graph, and it has no spare route if one of its roads closes.

2.6 Complexity comparison

Algorithm	Time	Space	Basis
Graph build (list)	O(V + E)	O(V + E)	dictionary of (neighbour, weight) lists; each road stored twice
Graph build (matrix)	$O(V^2)$	$O(V^2)$	V x V table, infinity where no road, 0 on the diagonal
Dijkstra	O((V + E) log V)	O(V + E)	min-heap of frontier nodes
Bellman-Ford	O(V x E)	O(V)	V - 1 sweeps over all roads, one more pass detects cycles
BFS	O(V + E)	O(V)	queue and visited set
DFS	O(V + E)	O(V)	recursion and visited set
Prim	O(E log V)	O(V + E)	heap of frontier roads
Kruskal	O(E log E)	O(V + E)	sort dominates, union-find near constant

The measured growth exponents in section 4 lie between 0.86 and 1.09 against E, close to the theoretical value of 1. Bellman-Ford keeps a V x E worst case, but its early exit keeps these random-graph runs near linear.

3. Interactive application

`app.py` is a Streamlit web dashboard and `cli.py` is a menu-driven command-line interface. Both sit on top of the same `algorithms/` package as the experiments, so the numbers shown in the interface match the terminal runs and the tables in this report. They make the tools usable by non-programmers: planners can pick intersections, slow down roads, and read routes, traversal orders and spanning trees without touching the code.

Run the interfaces:

```
streamlit run app.py
python cli.py
```

[Insert screenshots of the Streamlit dashboard here: route planning, congestion comparison, connectivity and MST views.]

4. Experiments

Each size V builds a seeded random connected graph: a random spanning tree first, then extra random roads, with weights 1 to 20. Density grows with V (E = max(1.8 V, 0.1316 V^2), capped at V(V - 1)/2), so E rises from 18 at V = 10 to 2,960 at V = 150. Every algorithm runs from the same first node on the same graph. Timing is the best of 15 runs (best of 5 for Bellman-Ford); peak memory comes from one separate tracemalloc run per size, so tracing overhead does not inflate the times.

analysis/benchmark.py (benchmark helpers):

```
import random, time, tracemalloc
def random_graph(V):
    """Connected random graph; E = max(1.8 V, 0.1316 V^2) capped at V(V-1)/2."""
    E = min(max(int(1.8 * V), int(0.1316 * V * V)), V * (V - 1) // 2)
    edges = {}
    for i in range(1, V):                       # random spanning tree first
        edges[(random.randrange(i), i)] = random.randint(1, 20)
    while len(edges) < E:                       # then extra random roads
        a, b = random.sample(range(V), 2)
        edges.setdefault((min(a, b), max(a, b)), random.randint(1, 20))
    return [(str(a), str(b), w) for (a, b), w in edges.items()]
def best_time_ms(fn, reps=15):
    best = float("inf")
    for _ in range(reps):
        start = time.perf_counter()
        fn()
        best = min(best, time.perf_counter() - start)
    return best * 1000
def peak_memory_kb(fn):
    tracemalloc.start()
    fn()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024
```

```
$ python analysis/benchmark.py
RUNNING CAPSTONE GRAPH-ALGORITHM BENCHMARKS (seed 42)
-> Dijkstra
  V= 10, E=  18: 0.00617 ms, peak 0.5 KB
  V= 25, E=  82: 0.02154 ms, peak 2.0 KB
  V= 50, E= 329: 0.07134 ms, peak 3.8 KB
  V= 75, E= 740: 0.14689 ms, peak 4.0 KB
  V=100, E=1316: 0.23792 ms, peak 8.0 KB
  V=125, E=2056: 0.35648 ms, peak 8.5 KB
  V=150, E=2960: 0.51267 ms, peak 8.8 KB
-> Bellman-Ford
  V= 10, E=  18: 0.00461 ms, peak 0.3 KB
  V= 25, E=  82: 0.03598 ms, peak 1.2 KB
  V= 50, E= 329: 0.14319 ms, peak 2.3 KB
  V= 75, E= 740: 0.33613 ms, peak 2.3 KB
  V=100, E=1316: 0.57599 ms, peak 4.7 KB
  V=125, E=2056: 0.88315 ms, peak 4.7 KB
  V=150, E=2960: 1.37615 ms, peak 4.7 KB
... Prim, Kruskal, BFS and DFS follow in the same format ...
```

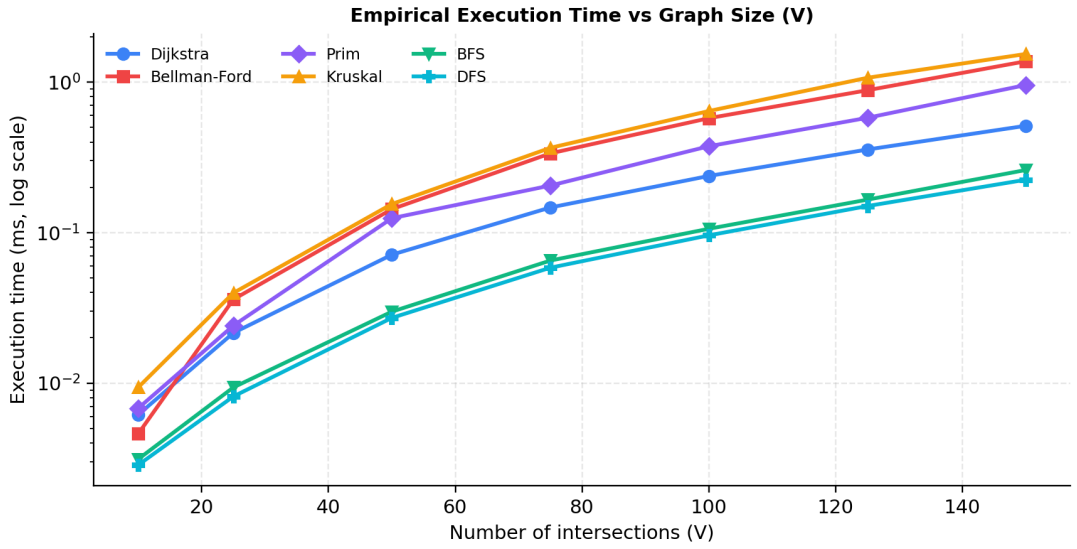
Execution time in milliseconds (random graphs, seed 42):

V	E	Dijkstra	Bellman-Ford	Prim	Kruskal	BFS	DFS
10	18	0.006	0.005	0.007	0.009	0.003	0.003
25	82	0.022	0.036	0.024	0.040	0.009	0.008
50	329	0.071	0.143	0.125	0.155	0.030	0.027
75	740	0.147	0.336	0.205	0.366	0.065	0.058

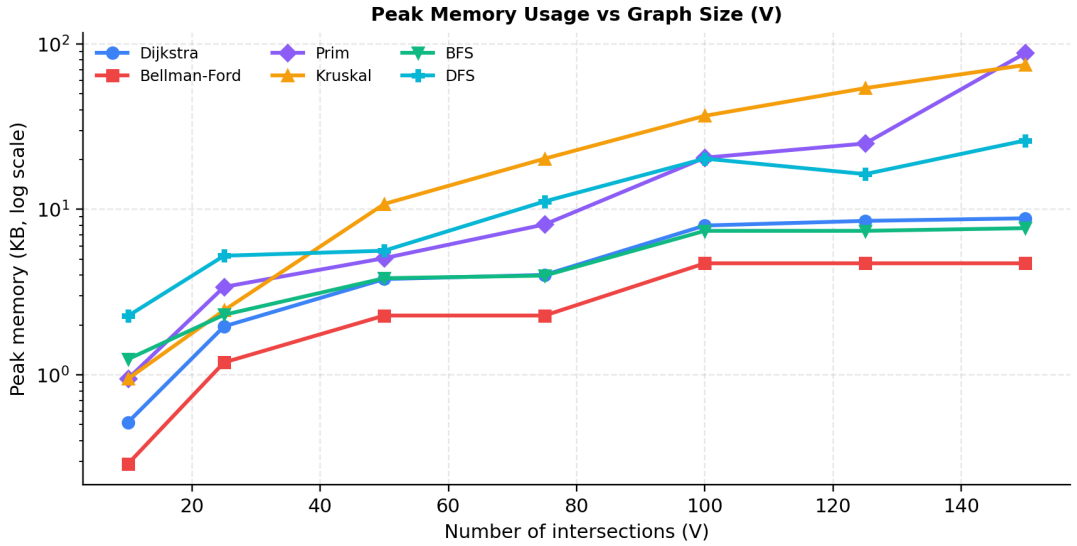
V	E	Dijkstra	Bellman-Ford	Prim	Kruskal	BFS	DFS
100	1316	0.238	0.576	0.375	0.643	0.106	0.096
125	2056	0.356	0.883	0.579	1.068	0.166	0.150
150	2960	0.513	1.376	0.957	1.539	0.261	0.225

Peak memory in KB (single traced run per size):

V	E	Dijkstra	Bellman-Ford	Prim	Kruskal	BFS	DFS
10	18	0.5	0.3	0.9	0.9	1.2	2.3
25	82	2.0	1.2	3.4	2.5	2.3	5.2
50	329	3.8	2.3	5.1	10.8	3.8	5.6
75	740	4.0	2.3	8.1	20.2	4.0	11.2
100	1316	8.0	4.7	20.5	36.8	7.4	20.3
125	2056	8.5	4.7	25.0	54.0	7.4	16.3
150	2960	8.8	4.7	88.1	74.6	7.7	26.1



Execution time versus graph size (logarithmic time axis).



Peak memory versus graph size (logarithmic memory axis).

- **Traversals are cheapest.** BFS (0.26 ms) and DFS (0.22 ms) touch each road once at V = 150.
- **Memory.** At V = 150 Bellman-Ford used the least (4.7 KB: one distance table) and Prim the most (88.1 KB: frontier heap plus visited set and tree). Single-run memory readings are lumpy, so read the trend and not single points.
- **Measurement noise.** Runs below one millisecond are noisy; differences between algorithms of similar cost are within measurement error.

4.1 BFS vs DFS

V	BFS (ms)	DFS (ms)	BFS / DFS	BFS (KB)	DFS (KB)
10	0.003	0.003	1.00	1.2	2.3
25	0.009	0.008	1.12	2.3	5.2
50	0.030	0.027	1.11	3.8	5.6
75	0.065	0.058	1.12	4.0	11.2
100	0.106	0.096	1.10	7.4	20.3
125	0.166	0.150	1.11	7.4	16.3
150	0.261	0.225	1.16	7.7	26.1

Both are linear in V + E. From V = 25 upward DFS is about 10 to 16 percent faster in these runs, but the gap is small enough to sit near measurement noise. DFS used more traced memory at every size, which is expected from its recursion and nested helper, while BFS only keeps a queue and a level map.

4.2 Dijkstra vs Bellman-Ford

V	Dijkstra (ms)	Bellman-Ford (ms)	Ratio (BF / Dijkstra)
10	0.006	0.005	0.8x
25	0.022	0.036	1.6x

V	Dijkstra (ms)	Bellman-Ford (ms)	Ratio (BF / Dijkstra)
50	0.071	0.143	2.0x
75	0.147	0.336	2.3x
100	0.238	0.576	2.4x
125	0.356	0.883	2.5x
150	0.513	1.376	2.7x

At the smallest size Bellman-Ford is marginally faster, because there is almost nothing to relax and it exits early. From V = 25 upward Dijkstra wins and the ratio widens steadily to 2.7x at V = 150, the price Bellman-Ford pays for accepting negative weights. Bellman-Ford stops once a pass changes nothing, so on random graphs it needs few passes; its worst case is still O(V x E).

4.3 Prim vs Kruskal

V	Prim (ms)	Kruskal (ms)	Ratio (Kruskal / Prim)	MST agrees?
10	0.007	0.009	1.29x	yes
25	0.024	0.040	1.67x	yes
50	0.125	0.155	1.24x	yes
75	0.205	0.366	1.79x	yes
100	0.375	0.643	1.71x	yes
125	0.579	1.068	1.84x	yes
150	0.957	1.539	1.61x	yes

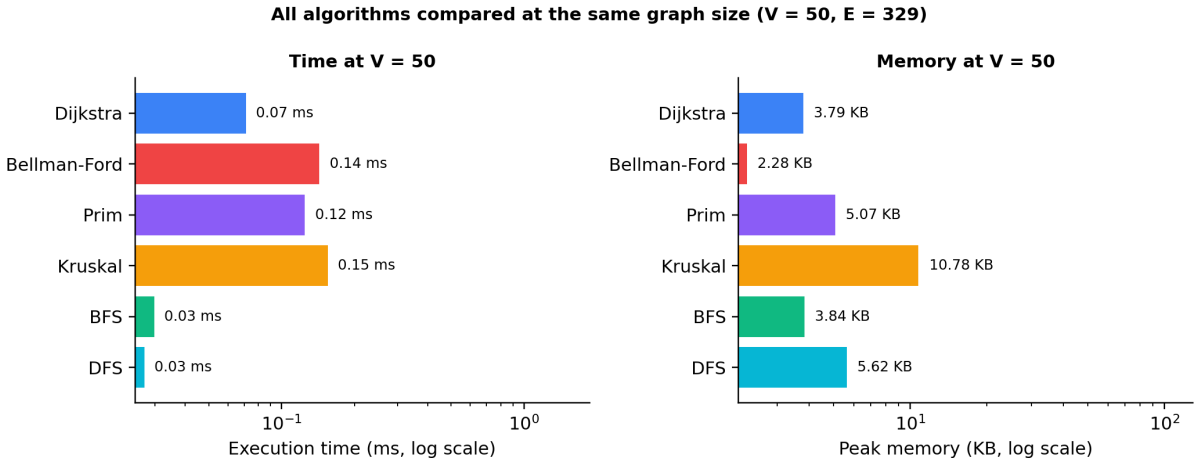
Prim is faster at every size (about 1.2x to 1.8x) because its heap holds only the frontier, while Kruskal sorts all E roads. Both agree on the MST cost, so either builder is safe and the pair cross-validates the result (23 on the city graph). On sparse city networks the two cost about the same.

4.4 Observed growth

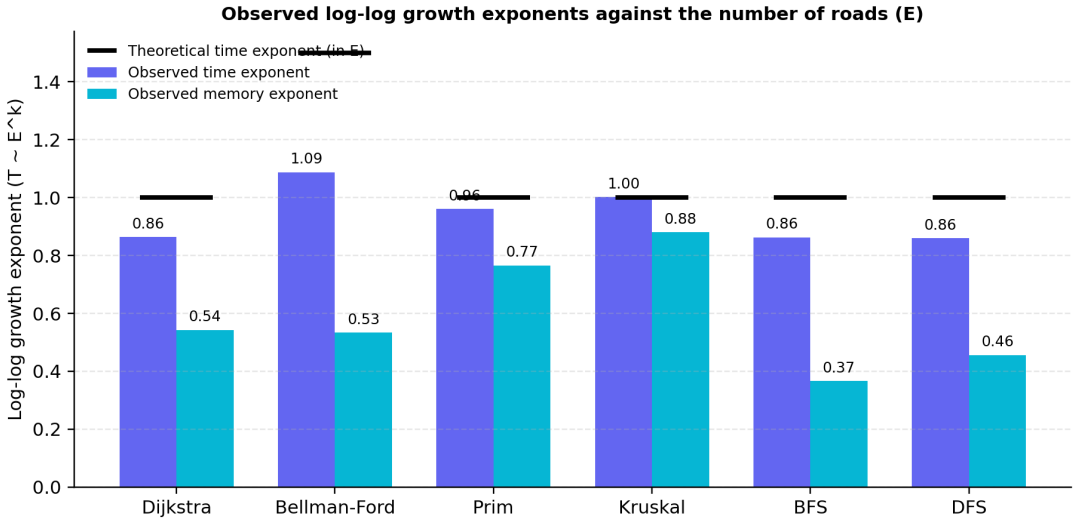
The observed slope is the gradient of log(time) against log(E), fitted over all seven sizes by least squares; a slope k means time grows as E^k. Because E grows roughly as V^2 in these graphs, the linear-in-E algorithms are expected near 1.

Algorithm	Theory time	Theory space	Time at V = 150 (ms)	Peak mem (KB)	Time slope (vs E)	Mem slope (vs E)
Dijkstra	O((V + E) log V)	O(V + E)	0.513	8.8	0.86	0.54
Bellman-Ford	O(V x E)	O(V)	1.376	4.7	1.09	0.53
Prim	O(E log V)	O(V + E)	0.957	88.1	0.96	0.77
Kruskal	O(E log E)	O(V + E)	1.539	74.6	1.00	0.88
BFS	O(V + E)	O(V)	0.261	7.7	0.86	0.37
DFS	O(V + E)	O(V)	0.225	26.1	0.86	0.46

Algorithm	Growth trend observed (V grows 15x, E grows 164x)
Dijkstra	time grows 83x (slope 0.86 against E) and memory 17x (slope 0.54)
Bellman-Ford	time grows 299x (slope 1.09 against E) and memory 16x (slope 0.53)
Prim	time grows 141x (slope 0.96 against E) and memory 93x (slope 0.77)
Kruskal	time grows 164x (slope 1.00 against E) and memory 79x (slope 0.88)
BFS	time grows 83x (slope 0.86 against E) and memory 6x (slope 0.37)
DFS	time grows 79x (slope 0.86 against E) and memory 12x (slope 0.46)



All algorithms measured at the same graph size (V = 50, E = 329).



Observed log-log growth exponents against the number of roads E (black bars = theoretical time exponent).

Every time exponent lies between 0.86 and 1.09. The slopes are fitted to seven sizes from one run each, so they carry some noise, and fixed per-call overhead at small V pulls them slightly below the ideal values. The theoretical time slope of Bellman-Ford against E is between 1 (few passes, as here) and 1.5 (V passes with E proportional to V^2). The measured memory slopes sit below 1 for the O(V) algorithms because V grows more slowly than E.

5. Testing

The repository contains automated tests in `tests/test_algorithms.py`. Alongside them, the key results in this report were re-checked by running the algorithm modules directly on the 10-intersection city; every check below passed:

- Prim and Kruskal return 9 edges and the same total (23)
- Dijkstra A -> D returns A, C, B, D with cost 8
- BFS and DFS reach 10 of 10 intersections with 9-edge trees
- Graph has one connected component
- Congestion: B-C x5 reroutes A to D from 8 to 9; D-E x4 leaves A to G unchanged
- Bellman-Ford: one-way credit gives dist(I) = -2 and no cycle
- Bellman-Ford: two-way credit is reported as a negative cycle
- Adjacency matrix is symmetric with a zero diagonal
- MST saving (78 - 23) / 78 = 70.5 percent

```
$ python demo.py   # verification checks
ok   Prim and Kruskal return 9 edges and the same total (23)
ok   Dijkstra A -> D returns A, C, B, D with cost 8
ok   BFS and DFS reach 10 of 10 intersections with 9-edge trees
ok   Graph has one connected component
ok   Congestion: B-C x5 reroutes A to D from 8 to 9; D-E x4 leaves A to G unchanged
ok   Bellman-Ford: one-way credit gives dist(I) = -2 and no cycle
ok   Bellman-Ford: two-way credit is reported as a negative cycle
ok   Adjacency matrix is symmetric with a zero diagonal
ok   MST saving (78 - 23) / 78 = 70.5 percent
Checks passed: 9 / 9
```

[Paste the output of `python -m pytest tests/test_algorithms.py -v` *from the repository here.]*

6. Analysis

Theory and experiment agree. Dijkstra, BFS, DFS, Prim and Kruskal grow close to linearly in E, and Bellman-Ford sits between 1 and 1.5, matching O((V + E) log V), O(V + E), O(E log V), O(E log E) and O(V x E). The right algorithm depends on the question: Dijkstra is the fastest exact router for positive travel times, but its greedy choice breaks with negative weights, where Bellman-Ford is needed. BFS answers hop-count questions that Dijkstra does not, and the MST answers a cost question that neither shortest-path algorithm addresses.

Sparse versus dense: Prim's heap beat Kruskal's sort on the dense benchmark graphs, while on sparse city networks the two cost about the same; both agreed on the MST cost (23) in every check. Representation trade-off: the list is smaller for sparse graphs and the matrix gives O(1) road lookups; in this Python implementation the matrix was smaller once the graph became dense. The MST is a backbone, not a router: the tree route from A to D is 10 against 8 on the full network, and a tree has no spare routes when a road closes.

Algorithm	Time	Space	Result in this project
Adjacency list / matrix	O(V + E) / O(V^2)	O(V + E) / O(V^2)	Sparse city graph: list preferred
Dijkstra	O((V + E) log V)	O(V + E)	A to D: 8 min; 9 min after B-C slows 5x
Bellman-Ford	O(V x E)	O(V)	Handles one-way toll credits; detects cycles
BFS / DFS	O(V + E)	O(V)	All 10 intersections within 3 hops of A
Prim / Kruskal	O(E log V) / O(E log E)	O(V + E)	MST cost 23 vs 78: 70.5% less

- At the same size (V = 50, E = 329) the order from fastest to slowest is DFS, BFS, Dijkstra, Prim, Bellman-Ford, Kruskal. Bellman-Ford costs 2.7x Dijkstra at V = 150, the price of accepting negative weights.
- All runs are cheap at these sizes (largest Bellman-Ford run about 1.4 ms), so the choice of algorithm is driven by correctness requirements (negative weights, spanning trees, hop counts) more than by speed.
- Real-world uses: turn-by-turn navigation and congestion rerouting (Dijkstra), congestion pricing with credits (Bellman-Ford), emergency dispatch audits (BFS/DFS) and road-maintenance budgeting (MST).

7. Limitations

- **One random graph per size:** each timing is the best of repeated runs on a single graph, so small differences are within noise.
- **Memory** is a single tracemalloc run per size, which is lumpy for small structures, and the graph itself is built before tracing starts.
- **Synthetic data:** the benchmark graphs are random and denser than a real road network, and the city has only 10 intersections and 4 incident scenarios.
- **Bellman-Ford uses early exit,** so its measured growth is below its worst case; a graph built to need V - 1 passes would be slower.
- **Language and machine:** results come from CPython on one computer; absolute values will differ elsewhere, and the benchmark figures in the repository's own CSV may differ from this report.
- **Static model:** weights do not change over time and roads are two-way; live traffic data, one-way roads and lane capacities are not modelled.

8. Conclusion

The capstone models a city as a weighted graph and applies six classic algorithms to four practical questions. Dijkstra routes vehicles, and the congestion simulator shows rerouting saving up to 11 minutes per trip when a road is slowed. Bellman-Ford supports one-way toll credits and detects cycles. BFS shows that every intersection is within 3 hops of Downtown and identifies C, D and F as the best dispatch sites. Prim and Kruskal agree on a spanning tree of cost 23 against 78 for the full network, a 70.5% maintenance reduction.

The benchmarks confirm the expected growth of every algorithm, show that Dijkstra is 2.7x faster than Bellman-Ford at V = 150, and show that representation and algorithm choice matter. For city planners: use Dijkstra for live routing and reroute only when the active route is hit; build the maintenance plan with an MST verified by both Prim and Kruskal; and use BFS hop counts to place emergency services. Next steps: live traffic weights, one-way roads and lane capacity.

9. How to reproduce

```
python analysis/benchmark.py
python -m pytest tests/test_algorithms.py -v
streamlit run app.py
python cli.py
```

Benchmark results are exported to `data_csv/benchmark_results.csv` and figures to `graphes/` (seed 42).

Repository: https://github.com/akash77007/Smart-City-Traffic-Optimization